

PSPD – Programação p Sistemas Paralelos e Distribuídos, Prof.: Fernando W. Cruz

## Projeto de pesquisa: Monitoramento/observabilidade de aplicações em clusters K8S

### 1. Objetivos:

O Kubernetes (<https://kubernetes.io>) é uma plataforma interessante para o desenvolvimento e disponibilização de aplicações containerizadas por conta da sua flexibilidade no gerenciamento da infraestrutura provida para os serviços instanciados. O objetivo deste projeto é explorar as estratégias de monitoramento e observabilidade de aplicações baseadas em microsserviços em ambiente kubernetes, com foco na métrica de desempenho.

### 2. Requisitos para alcançar o objetivo proposto

Para atender ao que foi proposto, os alunos devem (i) escolher uma aplicação baseada em microsserviços, (ii) preparar o *framework* kubernetes em modo cluster, (iii) realizar testes de carga baseados em cenários previamente desenhados.

#### 2.1 Sobre a aplicação baseada em microsserviços

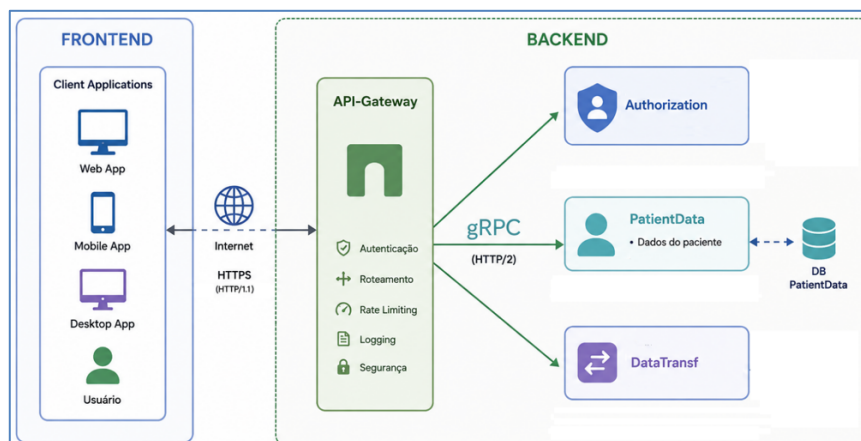
A aplicação baseada em microsserviços deve atender o Hospital Universitário que possui um grande volume de informações clínicas armazenadas em tabelas de um banco de dados tradicional. Essas informações podem ser disponibilizadas para usuários, sempre em formato HL7/FHIR<sup>1</sup>, que é um padrão para representar informações clínicas. Nele as informações são inseridas em estruturas chamadas Resources, que representam conceitos clínicos bem definidos, tais como: Patient → informações cadastrais do paciente; Encounter → consultas, internações e atendimentos; Condition → diagnósticos e doenças; Observation → exames laboratoriais e sinais clínicos; MedicationRequest → prescrições e medicamentos. O tipo de informação HL7/FHIR é liberado de acordo com os perfis listados a seguir:

- Médicos – podem acessar os dados do prontuário eletrônico sem qualquer tipo de anonimização: lista de pacientes sob sua responsabilidade, resumo clínico, histórico clínico, exames laboratoriais e medicamentos<sup>2</sup>. Acesso FULL aos dados.
- Estagiários – podem acessar dados de pacientes ligados ao médico supervisor (lista de pacientes supervisionados, resumo clínico, exames e medicamentos). No entanto, deve haver anonimização (remoção ou camuflagem) de informações pessoais (CPF, CNS, endereço completo, telefone e outros identificadores diretos). Acesso PARTIAL aos dados.
- Pesquisadores – podem acessar apenas (i) coortes (conjunto de pacientes que satisfazem um determinado critério clínico como Diabetes, Pneumonia, etc.), (ii) estatísticas agregadas (Exemplo: nesse coorte há um total de 14 mil casos, sendo 30% Homens, 70% Mulheres; Faixa etária: 18-39 anos 12%, 40-59 anos 44%, etc., Departamentos mais usados: Endocrinologia 33%, Cardiologia 15%, etc.), (iii) exames laboratoriais por paciente com dados anonimizados (Exemplo: paciente: hash001, sexo: F, idade: 63, Exames: HbA1c=8.1, glicemia=182, IMC=31.2; paciente hash002, sexo: M, idade: 58, Exames: HbA1c=7.2, glicemia=150, IMC=28.4, etc.), e (iv) informações relacionadas aos projetos de pesquisa (quais projetos possui e *status* de cada um). Acesso ANONYMIZED ou AGGREGATED aos dados.

A arquitetura da aplicação deve ser formada por um *frontend* e um *backend*, composto por uma API Gateway e três microsserviços, como ilustrado na Figura 1.

<sup>1</sup> Mais detalhes no endereço <https://www.hl7.org/fhir/>.

<sup>2</sup> Um resumo clínico é um relatório constando dados do paciente, diagnósticos principais (Diabetes, Hipertensão, etc.), dados do último atendimento (tipo de atendimento, setor que o atendeu e data do atendimento), últimos exames realizados e os valores (Glicemia: 182 mg/dL, Pressão arterial: 150/95 mmHg, etc.) e medicamentos em uso (Losartana 50 mg, Metformina 850 mg, etc.). Histórico clínico é um relatório temporal dos atendimentos do paciente, envolvendo condição clínica, medicamentos e resultados de exames. Exemplo: em 10/02/2023 foi diagnosticado com Diabetes Tipo 2, em 10/02/2023 foi medicado com Metformina 850 mg, em 18/04/2024 diagnosticado com Hipertensão Arterial, etc. Exames laboratoriais e Medicamentos é histórico clínico considerando apenas um tipo de evento, como resultados de exames ou medicamentos.



**Figura 1** – Arquitetura da aplicação baseada em microserviços

- O **frontend** é responsável pela interação com o usuário e, para isso, deve validar *login* e senha em um servidor OAuth2/OpenID Connect (<https://oauth.net/2/>), como o servidor Keycloak (<https://www.keycloak.org/>) ou similar, para obter um *token* JWT (JSON Web Token), que contém informações sobre o usuário autenticado (*username* e *role*, que pode ser médico, estagiário ou pesquisador). Esse *token* deve ser enviado em todas as chamadas para a API Gateway e, em função das respostas do *backend*, é montada a tela de consulta no *frontend*.
- A **API Gateway** é responsável por (i) receber requisições REST; (ii) validar tokens JWT; (iii) encaminhar requisições aos demais serviços; (iv) consolidar respostas apresentando-as aos usuários.
- O **Authorization Service** é responsável por verificar permissões do usuário em função do token JWT (ex.: *username* = med.cardoso, *role* = MEDICO), validar escopo da consulta e determinar o nível de acesso. Por exemplo, um médico com tipo de consulta = ResumoClínico do paciente P000001 ou um pesquisador com tipo de consulta = ResumoCoorte ligado ao projeto PRJ01. As seguintes autorizações de acesso são admitidas:
  - FULL – apresentar nome completo, data de nascimento, sexo, cidade, estado, CPF, CNS, diagnósticos, exames, medicamentos, atendimentos.
  - PARTIAL – apresentar iniciais do nome, sexo, faixa etária ou ano de nascimento, cidade/estado, diagnósticos, exames, medicamentos.
  - ANONYMIZED – apresentar identificador pseudonimizado, sexo, faixa etária, estado, condições clínicas, exames, medicamentos; remover: nome, CPF, CNS, cidade, data de nascimento exata, *patient\_id* real.
  - AGGREGATED – apresentar valores totalizados como total de pacientes, percentuais, médias, medianas, distribuições, contagens e frequências.

Com base nas autorizações é feita verificação nas tabelas do banco e tomada uma decisão, da seguinte forma: (i) O médico só pode acessar pacientes vinculados a ele. Se OK, autorização = ALLOW + FULL, caso contrário DENY; (ii) O estagiário só pode acessar pacientes vinculados a uma atividade supervisionada. Se existir esse vínculo, autorização = ALLOW + PARTIAL, caso contrário DENY; (iii) O pesquisador só pode acessar consultas de coorte associadas a projetos aprovados. Se o projeto estiver aprovado e vigente, autorização = ALLOW + ANONYMIZED ou ALLOW + AGGREGATED, caso contrário DENY.

- O **Patient Data Service** é responsável por consultas ao banco de dados (executar consultas SQL nas tabelas do pseudo-prontuário eletrônico tais como localizar os pacientes associados a um médico; localizar os pacientes supervisionados por um estagiário; recuperar os atendimentos de um paciente; recuperar diagnósticos, exames e medicamentos; recuperar os pacientes pertencentes a uma determinada coorte de pesquisa, agregações (quantidade de pacientes diabéticos; distribuição por sexo; distribuição por faixa etária; média da hemoglobina glicada; frequência de utilização de medicamentos; etc.); montagem dos dados clínicos a partir das seguintes tabelas:

| Tabela                   | Informações (colunas da tabela)                                                                                                                                                                                                                                                                                                                                 |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| patients                 | id_paciente, nome, data de nascimento, gênero, cidade, estado, cpf e número do CNS (cartão nacional de saúde)                                                                                                                                                                                                                                                   |
| encounters               | id_atendimento, id_paciente, data_inicio, data_fim, tipo de atendimento (Ambulatorial, Emergência, Internação, Retorno, etc.) e setor/departamento (Cardiologia, Endocrinologia, Pediatria, etc.)                                                                                                                                                               |
| clinical_events          | id_evento, id_paciente, id_atendimento, tipo do evento (pode ser Condição clínica, Observação obtida em exames laboratoriais ou Medicação), código do tipo de evento (Diabetes, Hipertensão, Creatinina, Insulina, Losartan, ...), descrição do evento, data_evento, valor e unidade do valor (campos preenchidos quando tipo_evento = Medicação ou Observação) |
| user_patient_assignments | id_vinculo, username do cuidador, id_paciente, tipo de vínculo do cuidador com o paciente (pode ser médico ou estagiário), username do supervisor do estágio e status do vínculo                                                                                                                                                                                |
| projects                 | id_projeto, título, username do pesquisador, código da condição clínica (Diabetes, Hipertensão, Obesidade, etc. – é igual ao código do tipo de evento da tabela clinical_events), status do projeto (Aprovado, Expirado, Suspenso, etc.) e data de validade do projeto.                                                                                         |

- O **Data Transform Service** é responsável por serviços de anonimização, agregação e transformação dos dados em formato HL7/FHIR, antes de serem entregues aos usuários. A seguir um mapeamento entre as tabelas do sistema e os recursos HL7/FHIR:

| Tabela PostgreSQL                      | Recurso HL7/FHIR  |
|----------------------------------------|-------------------|
| Patients (tabela de pacientes)         | Patient           |
| Encounters (atendimentos do paciente)  | Encounter         |
| clinical_events (Condição clínica)     | Condition         |
| clinical_events (Observação de exames) | Observation       |
| clinical_events (Medicações)           | MedicationRequest |

Por exemplo, um registro da tabela *patients*:

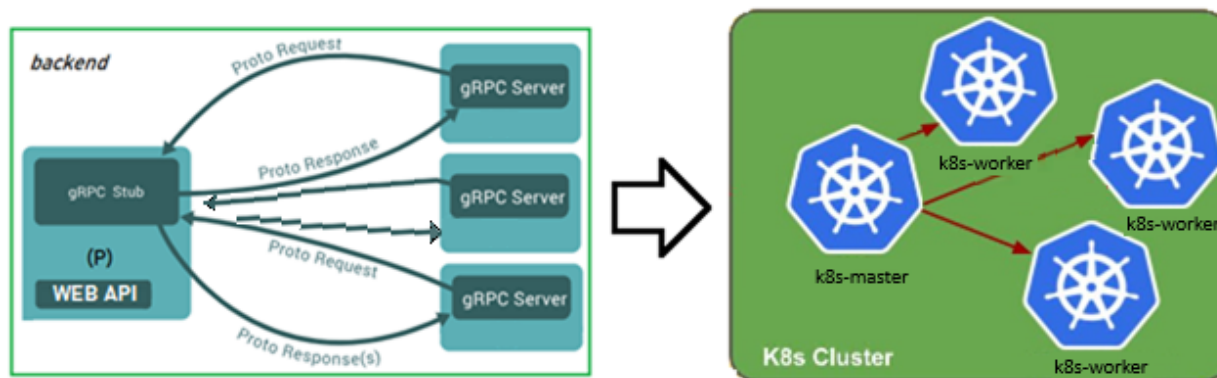
| patient_id | full_name     | birth_date | gender |
|------------|---------------|------------|--------|
| P000001    | João da Silva | 1970-05-10 | male   |

será convertido para:

```
{
  "resourceType": "Patient",
  "id": "P000001",
  "name": [
    {
      "text": "João da Silva"
    }
  ],
  "birthDate": "1970-05-10",
  "gender": "male"
}
```

## 2.2 Sobre a preparação do *framework* kubernetes, da ferramenta de monitoramento e dos testes de carga

Para comportar a aplicação citada no item anterior, deve-se estruturar uma instalação kubernetes em modo *cluster*, composto por um nó mestre (plano de controle) e pelo menos três nós escravos (*worker nodes*), incluindo interface web de monitoramento do cluster e recursos de *autoscaling*, conforme ilustrado na parte direita da Figura 2.



**Figura 2 – Instancição da aplicação no cluster K8S**

- Todos os passos e ferramentas utilizados para a criação do cluster kubernetes devem ser documentados.
- Deve-se priorizar a compreensão dos mecanismos disponíveis no kubernetes que permitem fazer com que uma aplicação containerizada se adapte a diferentes demandas de uso (por exemplo, o mecanismo de autoscaling do K8S).
- Para compreensão do que significa monitoramento e observabilidade, sugere-se a leitura dos Capítulos 15 e 16 do livro utilizado como referência nessa especificação (Seção 5, a seguir) e veja se o que está proposto na Seção 3 se encaixa. Caso contrário, veja que outras mudanças poderiam ser propostas para viabilizar ou melhorar aspectos de monitoramento e observabilidade.
- Como ferramenta de análise da aplicação instanciada no K8S, deve-se estudar e instalar, no K8S, o Prometheus (<https://prometheus.io/>), uma ferramenta voltada para observar e monitorar o comportamento de aplicações em clusters kubernetes baseado em métricas. As informações de instalação e recursos utilizados devem ser incluídas no projeto, de forma resumida.

### 3. Metodologia para garantir observabilidade e monitoramento

A aplicação baseada em microserviços é modular e suas partes podem ser acomodadas de diferentes formas no K8S. A título de exemplo, pode-se considerar que a forma mais simples é considerar os módulos da Figura 1 executando num único contêiner, enquanto que a forma mais distribuída é aquela na qual cada parte da aplicação está num contêiner instanciado em um *worker node* diferente. Por outro lado, sabe-se que o K8S possui elasticidade – ou seja, mecanismos que conseguem acomodar as aplicações em execução a diferentes demandas garantindo performance aceitável, independentemente da quantidade de requisições sofridas pela aplicação. No entanto, dependendo da arquitetura da aplicação, nem todos os arranjos são admitidos e, em alguns casos, a aplicação pode não funcionar adequadamente.

A metodologia de trabalho para essa entrega deve ser feita considerando as seguintes atividades:

- Validação funcional** – Inicialmente, cada grupo deverá executar sua aplicação utilizando uma réplica de cada microserviço, uma instância do banco PostgreSQL. Nesse caso, o objetivo é validar funcionamento da aplicação (autenticação, anonimização, conversão HL7/FHIR, etc.)
- Testes de carga**: Após a validação funcional, realizar os testes de desempenho. Ferramentas sugeridas para são o k6 e/ou o Locust (<https://locust.io/>). Realizar testes considerando 10, 50, 100, 500 e 1000 usuários simultâneos. Em cada caso, considerar pelo menos 4 métricas tais como *throughput*, latência média, utilização de CPU, utilização de memória e taxa de erro.
- Escalabilidade horizontal** – Os grupos devem aumentar o número de réplicas dos serviços e observar o comportamento da aplicação. Por exemplo, cada um dos módulos (API e microserviços), passando de uma instância para três réplicas de cada um. Nesses casos, deve-se analisar o ganho de desempenho, a utilização dos nós do cluster, a distribuição dos pods do K8S e o impacto no banco de dados.
- Autoscaling** – Aqui os grupos devem configurar HPA (Horizontal Pod Autoscaler). Por exemplo, ajustar para mínimo uma e, no máximo 10 réplicas ou ver escala baseada em CPU. De qualquer modo,

esses testes devem demonstrar: (i) a criação automática de pods, (ii) a redistribuição da carga, (iii) a redução de latência, e (iv) os limites de escalabilidade.

- e. Observabilidade – Todos os serviços devem expor métricas para coleta pelo Prometheus e o Grafana será utilizado para visualização. Considerar pelo menos 5 métricas. Algumas dicas: requisições por segundo, latência, tempo de resposta por serviço, uso de CPU, uso de memória, quantidade de pods, número de consultas ao banco; erros HTTP; erros gRPC.

Em qualquer cenário de teste, é importante:

- Documentar os atributos/métricas que serão testados
- Uso do Prometheus para monitorar/observar a aplicação e o ambiente testado
- Uso de ferramenta de teste para submissão da aplicação a diferentes cargas de trabalho (demandas)
- Garantir as mesmas condições de teste de infraestrutura para os testes de modo a não contaminar os resultados
- Para cada cenário montado, fazer teste de carga, observar o comportamento da aplicação e anotar as conclusões

#### 4. Questões de Ordem

Para essa especificação valem as seguintes regras:

- O experimento pode ser feito por grupos de 4 a 5 alunos; não serão aceitos trabalhos individuais. Nesse caso, basta que um dos alunos do grupo faça a postagem das entregas no Moodle (arquivo zipado).
- A entrega é composta por (i) códigos, instruções de uso e todas as informações necessárias para esclarecimento e uso dos programas entregues, (ii) um relatório, cuja estrutura e conteúdo estão descritos a seguir, (iii) um vídeo gravado pelos membros participantes, com apresentação do projeto. Nesse caso, considerar uma média de 4 a 6 minutos por aluno para que possam demonstrar como participaram e conhecimentos adquiridos.
- As comparações de desempenho são válidas se o programa mantiver sua execução adequada durante as execuções.
- O projeto entregue deve seguir a especificação feita neste documento. No entanto, alterações podem ser propostas associadas a uma justificativa, que será analisada para ponderar a nota que será atribuída ao projeto entregue.
- Projetos baseados em outras propostas são aceitos desde que haja referência ao projeto original e indicação das alterações e promovidas pelo grupo.
- Os alunos podem realizar o experimento em qualquer plataforma, inclusive em equipamentos locais, mas devem estar preparados para demonstração da aplicação funcionando *in loco*.
- O relatório a ser entregue deve conter o máximo conjunto de informações sobre o experimento (textos explicativos, figuras, roteiros de instalação, arquivos de configuração, parâmetros usados, etc.), a fim de dar qualidade ao relatório. Mais especificamente o relatório deve conter os seguintes pontos:
  - Dados do curso, da disciplina/turma, data e identificação dos alunos participantes
  - Introdução – pequena descrição da solicitação feita e uma visão geral sobre o conteúdo do relatório
  - A metodologia utilizada (como cada grupo se organizou para realizar a atividade, incluindo um roteiro sobre os encontros realizados e o que ficou resolvido em cada encontro)
  - Uma seção sobre a experiência de montagem do Kubernetes em modo cluster
  - Uma seção sobre cada uma das cinco fases do projeto (validação funcional, testes de carga, testes de carga, escalabilidade horizontal, autoscaling e observabilidade). Onde necessário, considerar os cenários de teste com relatos do teste, resultados encontrados e uma síntese/conclusão associada.
  - Conclusão – texto conclusivo em função da experiência realizada, comentários sobre dificuldades e soluções encontradas. Ao final, cada membro do grupo abre uma subseção para comentários pessoais sobre a pesquisa, indicando as partes que mais trabalhou, aprendizados e uma nota de autoavaliação.
  - Referências utilizadas – cuidado para não utilizar materiais de terceiros sem a devida citação.

- Anexos (opcional) – com eventuais informações não apresentadas anteriormente, tais como arquivos de configuração, comentários sobre os códigos construídos, instruções de execução e informações adicionais para permitir replicação do laboratório pelo professor. Arquivos e informações adicionais que não puderem ser postadas no Moodle podem ser disponibilizadas via *GitHub*.
- O projeto será avaliado sob dois aspectos: (i) qualidade das entregas, e (ii) participação, envolvimento com o experimento (descritos no vídeo). Com relação à qualidade das entregas, a nota é proporcional aos resultados apresentados. Por exemplo, bons testes/descobertas, percepção de equilíbrio na distribuição das tarefas do projeto entre os membros do grupo, boa documentação (incluindo vídeo) contam positivamente para obtenção de uma boa nota.
- A nota emitida levará em conta os seguintes atributos/pesos: (i) 20% para qualidade das entregas (relatório, vídeo, etc.), (ii) 80% para o nível técnico e de exploração das solicitações feitas.
- Ponto extra para funcionalidades não solicitadas. Por exemplo, proposição de outras formas de monitoramento e observabilidade para a aplicação, tal como a montagem de um pipeline de observabilidade considerando outras métricas não discutidas aqui.

## 5. Referências

[1] Arundel, J. and Domingus, J. Cloud Native DevOps with Kubernetes – Building, Deploying and Scaling Modern Applications in the Cloud, O’Reilly, 2019.